

F42: Fortran coding guides

Asis Hallab, “el Bobito”

April 29, 2025

1 Memory Passing Behavior in Fortran (F42 Standard for Tensor Omics)

1.1 General Principles

In Fortran (standards 77–2018), arguments to subroutines and functions are, by default, passed **by reference** rather than by copying. This applies to scalars, arrays, and derived types.

Thus:

- Scalars (e.g., `integer`, `real`) are passed by reference.
- Arrays (e.g., `integer`, `dimension(:)`, `real`, `dimension(:, :)`) are passed by reference.
- Derived types are passed by reference.

1.2 When Copies Are Made

Despite the default pass-by-reference behavior, hidden copies may be generated by the compiler in the following situations:

- **Passing expressions:** e.g., call `process(a+b)` requires evaluation and temporary storage.
- **Passing strided array sections:** e.g., `array(1:10:2)` is non-contiguous and may trigger copying.
- **Passing non-contiguous slices:** particularly relevant for rows in a 2D array.

1.3 Tensor Omics DataTables: Column and Row Passing

Tensor Omics DataTables store typed data in 2D Fortran arrays of the form:

- `real_array(n_rows, n_real_columns)`
- `int_array(n_rows, n_int_columns)`
- `char_array(n_rows, n_char_columns)`

Accessing Columns:

- Example: `real_array(:, j)`
- Behavior: Passed by reference, **no copy made**.
- Reason: Slicing across the first index in column-major layout yields contiguous memory.

Accessing Rows:

- Example: `real_array(i, :)`
- Behavior: Typically triggers a hidden copy.
- Reason: Slicing across the second index accesses non-contiguous memory addresses, requiring packing into a contiguous temporary.

1.4 Summary Tables

1.4.1 General Fortran Behavior

Action	Example	Copy Made?
Pass full array	<code>array</code>	No
Pass contiguous slice	<code>array(:, j)</code>	No
Pass strided slice	<code>array(1:10:2, j)</code>	Yes
Pass expression	<code>array(:, j) * 2.0</code>	Yes

1.4.2 Tensor Omics DataTables Specifics

Access Type	Example	Copy Made?
Access full column	<code>real_array(:, j)</code>	No
Access full row	<code>real_array(i, :)</code>	Yes

1.5 Best Practice Recommendation for Tensor Omics

- Always design core functions to work with columns if possible.
- Avoid passing strided slices or expressions directly to subroutines.
- Treat rows carefully: copy manually if absolutely needed, otherwise prefer per-element processing.

1.6 Conclusion

Efficient memory handling is critical for Tensor Omics. Understanding Fortran's passing conventions allows maximization of speed, minimization of hidden temporaries, and preservation of F42 computational standards.